Reviewer Pack — Minimal Discrepancy Tensor Rank of Polynomial Functions vs B…

Entry 2c51c30824ff · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 12:30:20 UTC

Minimal Discrepancy Tensor Rank of Polynomial Functions vs BP_ReadTwice Circuit Size

Entry ID: `2c51c30824ff` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-22 12:30:20 UTC

1. Conjecture

Field A (mathematical branch): Free Probability Theory **Field B** (complexity object): Complexity Theory: BP_ReadTwice Circuit Complexity

Statement:

> ['For every polynomial function f over {0,1}^n with degree d, there exists a discrepancy tensor T_f of rank at most O(d log n) such that for any read-twice branching program P computing f, the output distribution of P satisfies Discrepancy(P) >= 0.5 * Min_Rank(T_f).', 'Equivalently, if there exists a polynomial function f with degree d and discrepancy tensor T_f of rank less than O(d log n), then any read-twice BP P computing f must have size at most 2^{O(n/d)}.']

Rationale (proposer's reasoning):

> ['Free probability theory provides tools for studying the non-commutative aspects of random matrices, which can be used to analyze the discrepancy and tensor rank of functions.', 'This conjecture suggests that the rank of a discrepancy tensor might provide a meaningful invariant for BP_readtwice complexity, potentially revealing new insights into the hardness of read-twice branching programs.']

Taxonomy category: `BP_READTWICE` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `2cf7dfc0691b555d`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For a polynomial function f with degree d, if the discrepancy tensor T_f has rank at most O(d log n), then the associated read-twice BP P must have Discrepancy(P) >= 0.5 * Min_Rank(T_f).

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'free probability theory' AND 'BP_ReadTwice circuit complexity' AND 'tensor rank' - 'discrepancy tensor' AND 'polynomial function' AND 'read-twice branching program' - 'complexity theory' AND 'minimal discrepancy tensor rank' AND 'circuit size BP_ReadTwice'

Top relevant hits considered: - [http://arxiv.org/abs/1102.2932v2] Applications of Monotone Rank to Complexity Theory - [http://arxiv.org/abs/1204.6522v3] Formal Groups, Witt vectors and Free Probability - [http://arxiv.org/abs/1403.8106v1] Recent advances on the log-rank conjecture in communication complexity - [http://arxiv.org/abs/1805.01816v2] Polynomials and tensors of bounded strength - [http://arxiv.org/abs/2412.14390v2] Using SimTeEx to simplify polynomial expressions with tensors - [http://arxiv.org/abs/2411.10363v3] Hammersley Point Sets and Inverse of Star-Discrepancy - [http://arxiv.org/abs/2006.02374v2] On tensor rank and commuting matrices - [http://arxiv.org/abs/0911.3482v5] Complexity of Networks (reprise)

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = 10
    d = 5

    # Generate a random polynomial function f over {0,1}^n with degree d
    variables = [f'x{i}' for i in range(n)]
    terms = []
    for _ in range(d):
        coeffs = [random.choice([0, 1]) for _ in range(n + 1)]
        term = sum(c * v if i > 0 else c for i, (c, v) in enumerate(zip(coeffs, variables)))
        terms.append(term)
    f = sum(terms)

    # Compute the discrepancy tensor T_f
```

```
    T_f = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            if i == j:
                T_f[i][j] = 1

    # Construct a BP P that computes f using its characteristic function and measure
    def bp_p(x):
        return f.subs({v: x[i] for i, v in enumerate(variables)})

    # Measure the output distribution of P for each instance and calculate Discrepancy(P)
    instances_tested = 100
    discrepancy_sum = 0
    for _ in range(instances_tested):
        x = [random.choice([0, 1]) for _ in range(n)]
        y = bp_p(x)
        discrepancy_sum += abs(y - 0.5)

    discrepancy_avg = discrepancy_sum / instances_tested

    # Correlate Min_Rank(T_f) with Discrepancy(P) for multiple instances to check if the conjectur
    min_rank_T_f = sum(1 for row in T_f if any(val != 0 for val in row))

    metric_value = discrepancy_avg
    conjecture_holds = discrepancy_avg >= 0.5 * min_rank_T_f
    counterexample = "" if conjecture_holds else "discrepancy < 0.5 * min_rank(T_f)"

    return {
        "metric_name": "Discrepancy(P)",
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19,

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value) ** 2 for result in results) /
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        first_failing_seed = next(seed for seed, result in enumerate(results) if not result["conje
        print(f"RESULT: FALSIFIED counterexample=\"discrepancy < 0.5 * min_rank(T_f)\" first_faili
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_547e2fb0.py", line 74, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_547e2fb0.py", line 29, in run_trial
    term = sum(c * v if i > 0 else c for i, (c, v) in enumerate(zip(coeffs, variables)))
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which prevents us from verifying the conjecture's conditions. | next: Re-run the test with proper error handling to ensure it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14409
2	preregistration	ollama_remote	glm4:latest	0	0	10171
3	novelty	ollama_remote	glm4:latest	0	0	8550
4	novelty	ollama_remote	glm4:latest	0	0	9131
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14773
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8483
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9397
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9933
9	judge	ollama_remote	glm4:latest	0	0	11963

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 96810 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/2c51c30824ff.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/2c51c30824ff.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/2c51c30824ff.tar.gz` (if generated)